

Chapter Six:

Remote Method Invocation

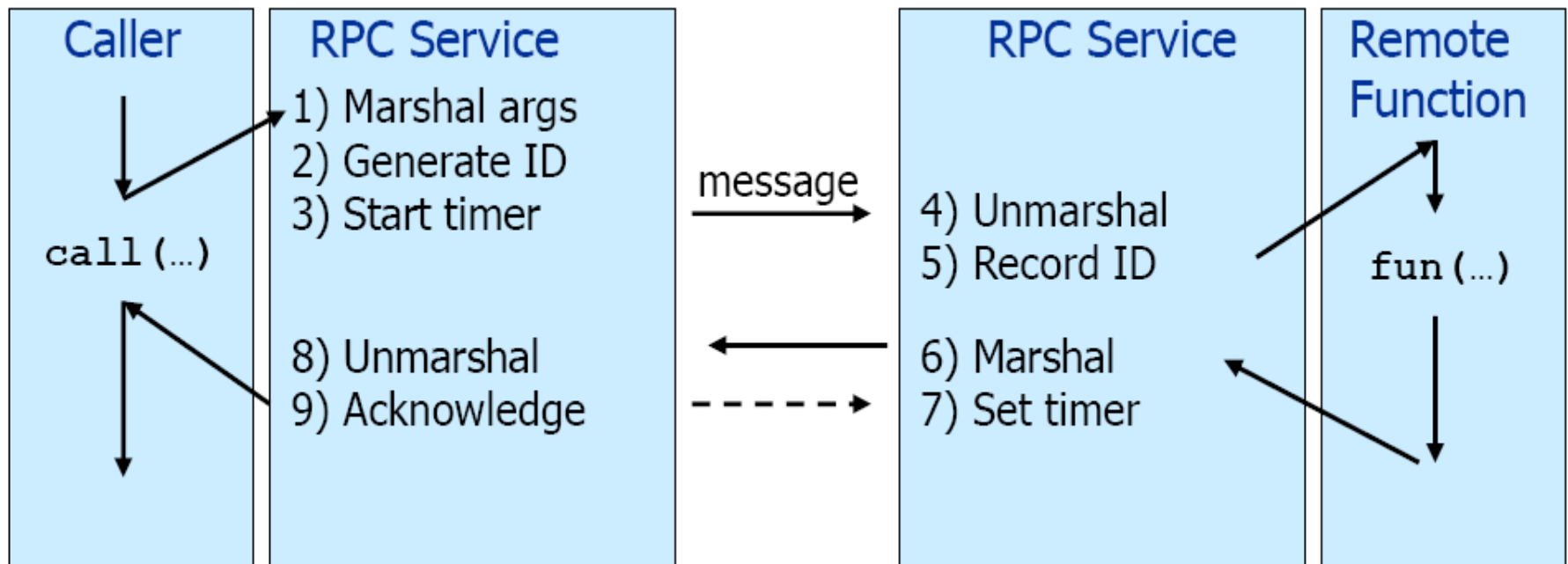
(RMI)

Remote Method Invocation (RMI)

- Covered in Java programming
- Distributed objects in Java

Overview of RMI

- RPC

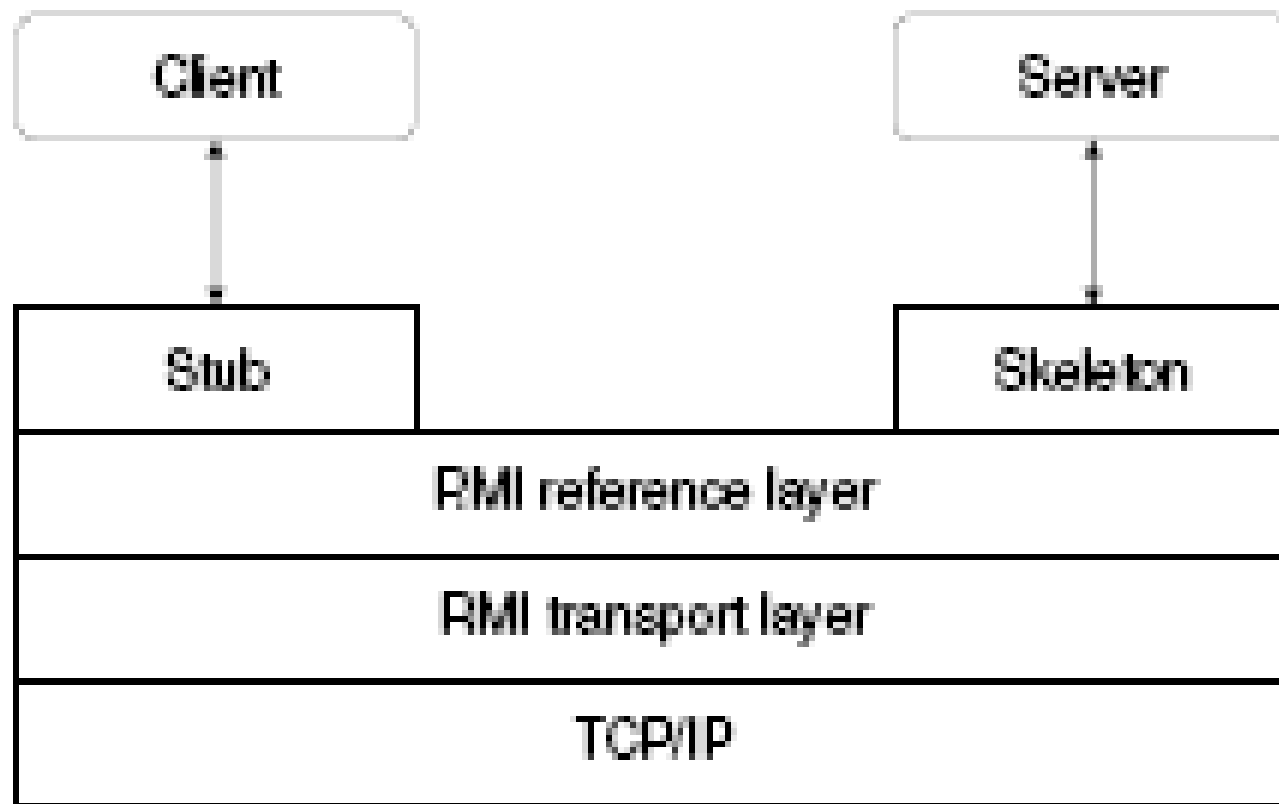


- RMI is just an object version of RPC

Overview of RMI

- RMI applications comprise two separate programs, a **server** and a **client**
 - The server program:
 - **creates** some remote objects
 - makes references to these objects accessible
 - waits for clients to invoke methods on these objects
 - The client program:
 - **obtains** a remote reference to one or more remote objects on a server
 - and then invokes methods on them

RMI Architecture



RMI Architecture

- The stub and skeleton layer in RMI
 - **Stub**
 - A placeholder object which offers the same interface as the server object
 - It does the following tasks
 - It initiates a connection with remote Virtual Machine (JVM),
 - It writes and transmits (marshals) the parameters to the remote Virtual Machine (JVM),
 - It waits for the result
 - It reads (unmarshals) the return value or exception, and
 - It finally, returns the value to the caller.

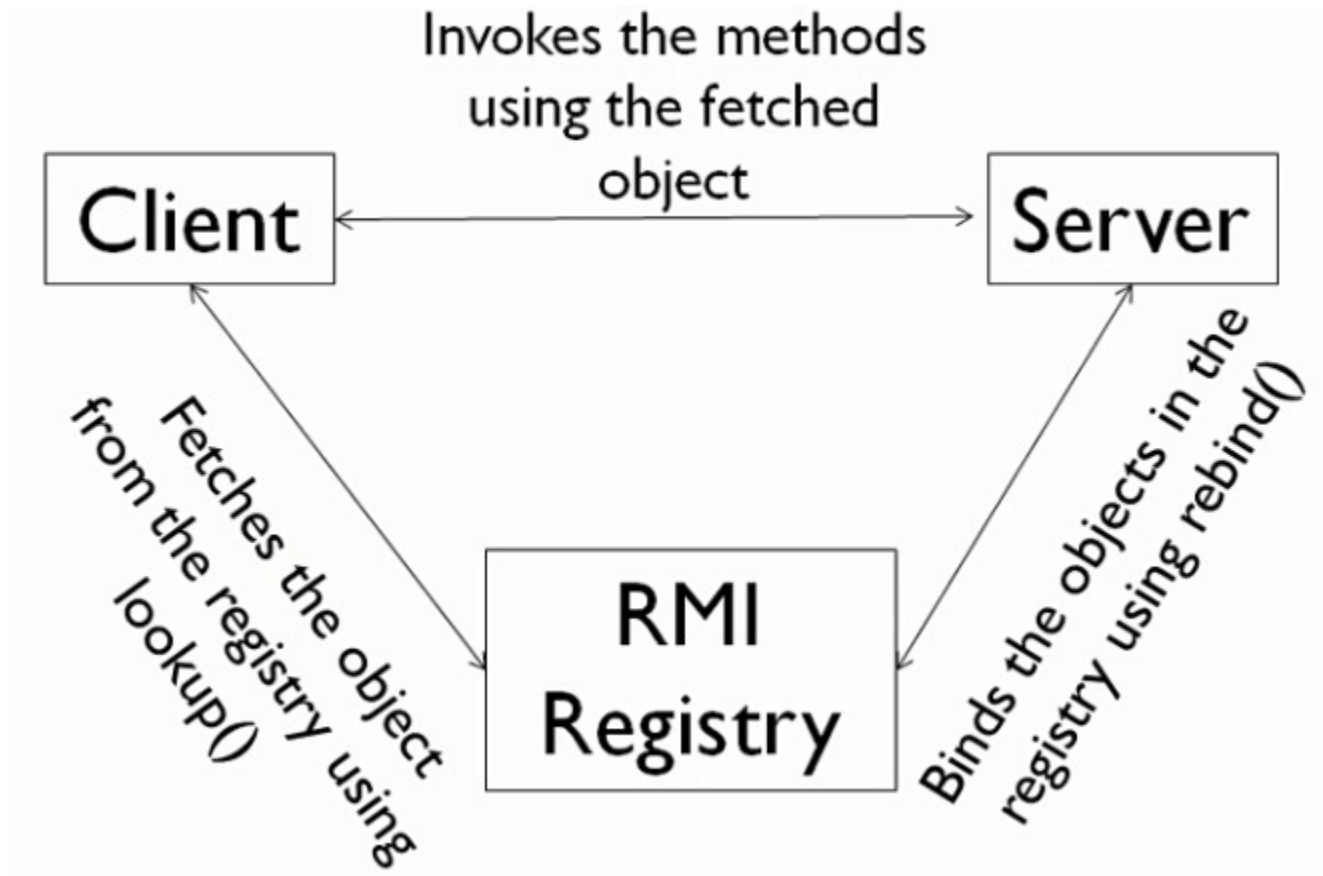
RMI Architecture ...

- Skeleton
 - The skeleton takes the calls of the stub
 - It processes them
 - It forwards the call to the server object
 - It waits for the result
 - It sends the result back to the stub
 - It does the following tasks
 - It reads the parameter for the remote method
 - It invokes the method on the actual remote object, and
 - It writes and transmits (marshals) the result to the caller.
- **NB: In the Java 2 SDK, an stub protocol was introduced that eliminates the need for skeletons.**

RMI Architecture

- The *reference layer* in RMI
 - It finds the respective communication partners
 - It includes the name service, the *registry*
- The transport layer in RMI
 - It manages connections
 - It handles communication
 - It must not be confused with the network transport layer (e.g. TCP/IP)

Process of RMI



RMI Packages and Classes

- Package *java.rmi*
 - Provides the RMI package
- Interface *java.rmi.Remote*
 - Identifies interfaces whose methods may be invoked from a non-local virtual machine
 - Any object that is a remote object must directly or indirectly implement this interface
 - Only those methods specified in a "remote interface" (an interface that extends *java.rmi.Remote*) are available remotely
- Class *java.rmi.RemoteException*
 - Communication-related exceptions that may occur during the execution of a remote method call
 - Each method of a remote interface (an interface that extends *java.rmi.Remote*) *must list RemoteException in its throws clause*

RMI Packages and Classes

- Package *java.rmi.server*
 - Provides classes and interfaces for supporting the server side of RMI
- Class *java.rmi.server.RemoteObject*
 - Distributed objects do not inherit directly from *Object*
 - They inherit from *RemoteObject*
 - *RemoteObject* implements the *Object* behavior for remote objects (e.g. methods *hashCode*, *equals*, and *toString* are reimplemented)
- Class *java.rmi.server.UnicastRemoteObject*
 - Provides support for point-to-point active object references (invocations, parameters, and results)
 - Uses TCP streams

Steps to Create RMI Applications

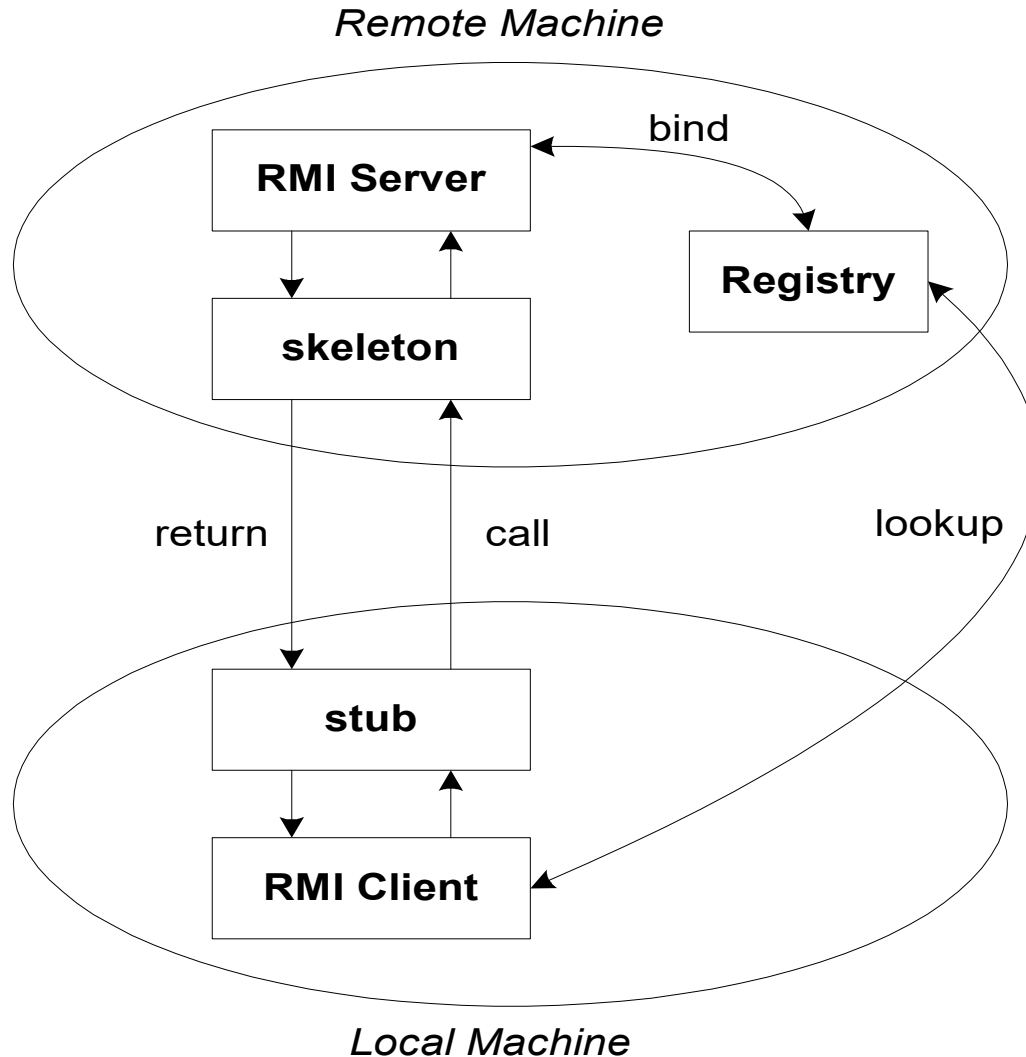
The following is set of six steps given to write the RMI program

1. Create the remote interface
2. Provide the implementation of the remote interface
3. Compile the implementation class and create the stub and skeleton objects using the **rmic** tool
4. Start the registry service by **rmiregistry** tool
5. Create and start the remote application
6. Create and start the client application

Steps to Create RMI Applications (2)

1. Define the Remote Interface
 - a) create its .jar file using Clean-Build
2. Develop Server program
 - a) include .jar file of remote interface
3. Develop client program
 - a) include .jar file of remote interface
4. Execute server program
5. Execute client program

General RMI Architecture



The Naming class provides methods to get and store the remote object. The Naming class provides 5 methods.

public static java.rmi.Remote lookup (java.lang.String) throws java.rmi.NotBoundException, java.net.MalformedURLException, java.rmi.RemoteException;	It returns the reference of the remote object.
public static void bind (java.lang.String, java.rmi.Remote) throws java.rmi.AlreadyBoundException, java.net.MalformedURLException, java.rmi.RemoteException;	It binds the remote object with the given name.
public static void unbind (java.lang.String) throws java.rmi.RemoteException, java.rmi.NotBoundException, java.net.MalformedURLException;	It destroys the remote object which is bound with the given name.
public static void rebind (java.lang.String, java.rmi.Remote) throws java.rmi.RemoteException, java.net.MalformedURLException;	It binds the remote object to the new name.
public static java.lang.String[] list (java.lang.String) throws java.rmi.RemoteException, java.net.MalformedURLException;	It returns an array of the names of the remote objects bound in the registry.

Steps ...

- Define the remote interface provided by the remote object:
 - Extends *java.rmi.Remote*
 - Defines the set of methods that can be called remotely
 - Each method must throw *java.rmi.RemoteException*
- Implement the remote object in a class:
 - Declare the remote interface being implemented
 - Implement the set of methods that can be called remotely
 - Implement any other local method that can not be invoked remotely

Steps ...

- Implement the remote object in a class – Passing objects in RMI
 - Arguments to remote methods or return values from remote methods can be of any type
 - Primitive data types (e.g. int, float, etc.)
 - Remote objects
 - Local objects
 - Objects passed to or returned from remote methods must be *serializable*
 - They must implement the *java.io.Serializable interface*
 - Some object types do not meet any of these criteria; they cannot be passed to or returned from remote methods
 - Most of these objects, such as threads or file descriptors, encapsulate information that makes sense only within a single address space
 - Many of the core classes (e.g. classes in the packages *java.lang* and *java.util*) implement the *Serializable interface*

Steps ...

- How arguments and return values are passed in remote method invocations
- Remote objects are essentially passed by reference
 - A remote object reference is a stub
 - It is a client-side proxy that implements the remote interface that the remote object implements
 - Passing a remote object by reference means that any changes made to the state of the object by remote method invocations are reflected in the original remote object
- Local objects are passed by copy
 - Using object serialization
 - By default, all fields are copied except fields that are marked static or transient
 - Default serialization behavior can be overridden on a class-by-class basis
 - A copy of the object is created in the receiving Java virtual machine
 - Any changes to the object's state by the receiver are reflected only in the receiver's copy, not in the sender's original instance
 - Any changes to the object's state by the sender are reflected only in the sender's original instance, not in the receiver's copy

Steps ...

- Implement the server program:
 - Create and install a security manager
 - Create and export remote objects
 - Register remote objects with the RMI registry

Steps ...

- Implement the server program – Create and install a Security Manager
 - The first task of the server program is to create and install a security manager
 - This protects access to system resources from untrusted downloaded code running within the Java virtual machine
 - A security manager determines whether downloaded code has access to the local file system or can perform any other privileged operations
 - If an RMI program does not install a security manager, RMI will not download classes (other than from the local class path) for objects received as arguments or return values of remote method invocations
 - This restriction ensures that the operations performed by downloaded code are subject to a security policy

Steps ...

- Implement the client program:
 - Create and install a security manager
 - Get a remote object reference
 - Perform remote method invocations on the remote object
- Compile source files
 - The remote interface
 - The remote object implementation class
 - The server program class
 - The client program class

Steps ...

- Run server-side and client-side programs:
- Server-side
 - Start RMI registry
 - Start the server
- Client-side
 - Start the client

Example

- **Step 1:**

- Define the interface

```
import java.rmi.Remote;
```

```
import java.rmi.RemoteException;
```

```
public interface Hello extends  
    Remote
```

```
{
```

```
    String sayHello() throws  
    RemoteException;
```

```
}
```

Step 2:

- Develop the remote object by implementing the remote interface
- The server is a simple unicast remote server.
- Create server by extending `java.rmi.server.UnicastRemoteObject`.
- The server uses the `RMI SecurityManager` to protect its resources while engaging in remote communication

```
public class Server extends UnicastRemoteObject  
    implements Hello {
```

```
    public Server() throws RemoteException    //  
        Constructor  
    {  
        super();  
    }
```

```
    ...
```

- Should give implementation for the interface method

```
public String sayHello()  
{  
    return "Hello, world!";  
}
```

Combining all together

```
import java.rmi.registry.Registry;
import java.rmi.registry.LocateRegistry;
import java.rmi.RemoteException;
import java.rmi.server.UnicastRemoteObject;
import java.rmi.*;
import java.security.*;

    public class Server extends UnicastRemoteObject
    implements Hello {
        public Server() throws RemoteException {}
        public String sayHello() {
            return "Hello, world!";
        }
    }
```

```
public static void main(String args[]) {  
    try {  
        Server obj = new Server();  
        //System.setSecurityManager(new SecurityManager());  
  
        // Bind the remote object's stub in the registry  
        Registry registry = LocateRegistry.createRegistry(3000);  
        registry.rebind("Hello", obj);  
        System.out.println("Server ready");  
    } catch (Exception e) {  
        System.err.println("Server exception: " +  
            e.toString());  
        e.printStackTrace();  
    }  
}
```

- **Step 3: Defining the client**

```
import java.rmi.registry.LocateRegistry;
import java.rmi.registry.Registry;
import java.rmi.*;
public class Client {
    private Client() {}
    public static void main(String[] args) {
        String host = "//127.0.0.1:3000/Hello";
        try {
            Registry registry = LocateRegistry.getRegistry(host);
            Hello stub = (Hello) Naming.lookup(host);
            String response = stub.sayHello();
            System.out.println("response: " + response);
        } catch (Exception e) {
            System.err.println("Client exception: " +
                e.toString());
            e.printStackTrace();
        } } }
```

To Run RMI Program

For running RMI programs use the following steps

1) compile all the java files that you have created for RMI In

one CMD i.e javac *.java

2) create stub and skeleton object by rmic tool

e.g. rmic CalcRemote in the same CMD to the above

3) start rmi registry in the same command prompt to the

above i.e. rmiregistry 3000

4) start the server in another command prompt

e.g java Server

5) start the client application in another command prompt